

Java Interview Prep

Free Sample

14 Q&A questions · 2 DSA mock problems · the 60-minute mock structure

For fresher & 0-2 year Software Engineer / SDE roles

PREP TREK BY JOBNETWORK · JOB-NET-WORK.VERCEL.APP/PREP-TREK/JAVA

Prep Trek: Java Interview Q&A (Free Sample)

How to use: Cover the answer, read the question, answer out loud in 30-60 seconds, then check your answer against the one below. Mark each question with a tick (said it cleanly), a question mark (partly), or a cross (missed), and redo the misses two days later.

Part 1: Core Java Basics

Q1. What is the difference between JDK, JRE, and JVM?

Show answer:

- **JVM:** runs bytecode; platform-specific; handles memory and garbage collection.
- **JRE:** JVM + core libraries needed to *run* Java programs.
- **JDK:** JRE + development tools (compiler `javac`, debugger, etc.) needed to *write and build* programs.

Relationship: $JDK \supset JRE \supset JVM$. "Write once, run anywhere" works because `javac` produces platform-independent bytecode, and each OS has its own JVM.

Q2. Why is Java platform independent?

Show answer:

Source code compiles to **bytecode**, not machine code. Any machine with a JVM can run that bytecode. The JVM, not your code, is platform-specific.

Follow-up: *Is the JVM platform independent?* No. Each OS needs its own JVM implementation.

Part 2: OOP in Java

Q3. Explain the four pillars of OOP with a Java example.

Show answer:

Use a payment system.

- **Encapsulation:** `private double balance;` with `getBalance()` and `deposit()`; validation lives inside the class.
- **Inheritance:** `class CreditCardPayment extends Payment`, reusing shared fields and methods.
- **Polymorphism:** `Payment p = new UpiPayment(); p.process();` calls the UPI version at runtime.
- **Abstraction:** `abstract class Payment { abstract void process(); }`; callers don't see implementation details.

Q4. Method overloading vs overriding?

Show answer:

	Overloading	Overriding
Where	Same class	Parent and child class
Signature	Different parameters	Same signature
Resolved	Compile time	Runtime
Return type	Can differ	Same or covariant

```

class Calc {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; } // overload
}
class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Woof"); } // override
}

```

You cannot override `static`, `final`, or `private` methods.

Q5. Abstract class vs interface?

Show answer:

	Abstract class	Interface
Inheritance	Extend one	Implement many
State	Can have instance fields	Only constants (<code>public static final</code>)
Methods	Abstract + concrete	Abstract, plus <code>default</code> and <code>static</code> (Java 8+)
Constructor	Yes	No
Use when	Shared base code ("is-a")	Capability contract ("can-do")

Part 3: Strings

Q6. Why is `String` immutable in Java?

Show answer:

Reasons:

1. **Security:** strings hold usernames, file paths, and URLs; mutation after validation would be dangerous.
2. **String pool:** literals can be safely shared only if nobody can change them.
3. **Thread safety:** immutable objects need no synchronization.
4. **Hash caching:** `hashCode()` is computed once, which makes strings good `HashMap` keys.

Q7. String vs StringBuilder vs StringBuffer?

Show answer:

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed	Slow for repeated edits	Fastest	Slower than StringBuilder

Use `StringBuilder` for string building in loops, and `StringBuffer` only if multiple threads share it.

Part 4: Exceptions

Q8. Checked vs unchecked exceptions?

Show answer:

- **Checked:** verified at compile time; you must handle or declare them (`IOException`, `SQLException`). They extend `Exception` but not `RuntimeException`.

- **Unchecked:** extend `RuntimeException` ; not forced by the compiler (`NullPointerException` , `ArrayIndexOutOfBoundsException`).
- **Errors** (`OutOfMemoryError` , `StackOverflowError`) are serious JVM problems that you generally don't catch.

Part 5: Collections

Q9. ArrayList vs LinkedList?

Show answer:

	ArrayList	LinkedList
Backing	Dynamic array	Doubly linked list
Get by index	O(1)	O(n)
Add/remove at end	O(1) amortized	O(1)
Insert/remove in middle	O(n) shifting	O(1) once node is found, but finding is O(n)
Memory	Lower	Higher (node pointers)

Default to `ArrayList` . In practice it wins in most cases because of cache-friendly memory layout.

Q10. How does HashMap work internally?

Show answer:

1. `key.hashCode()` is computed and spread to pick a **bucket index**.
2. If the bucket is empty, the entry is stored there.
3. If it has entries (a **collision**), keys are compared with `equals()` ; matching key → value replaced, otherwise the entry is added to the bucket's chain.
4. Since Java 8, a long chain in a bucket (8+ entries, with a large enough table) converts to a balanced tree, improving worst-case lookup from O(n) to O(log n).
5. When size exceeds `capacity × load factor` (default 0.75), the table **resizes** (doubles) and entries are redistributed.

Average `get / put` : O(1).

Follow-up: Why must you override `hashCode()` when you override `equals()` ? Equal objects must produce equal hash codes, or the map looks in the wrong bucket and can't find them.

Part 6: Multithreading

Q11. Two ways to create a thread? Which is better?

Show answer:

1. Extend `Thread` and override `run()` .
2. Implement `Runnable` and pass it to a `Thread` .

Prefer `Runnable` : Java has single inheritance, so extending `Thread` uses up your one chance, and `Runnable` separates the task from the mechanism. In real projects, use an `ExecutorService` instead of creating threads manually.

Also: call `start()` (creates a new thread), not `run()` (runs in the current thread like a normal method).

Part 7: Java 8+ Features

Q12. What are lambda expressions and functional interfaces?

Show answer:

A **functional interface** has exactly one abstract method (`Runnable` , `Comparator` , `Function`). A **lambda** is a short way to implement it:

```
Runnable r = () -> System.out.println("hello");
Comparator<Integer> c = (a, b) -> b - a;
```

Common built-ins: `Predicate<T>` (boolean test), `Function<T,R>` (transform), `Consumer<T>` (use a value), `Supplier<T>` (provide a value).

Q13. What is the Stream API? Give an example.

Show answer:

Streams process collections declaratively: **source** → **intermediate operations (lazy)** → **terminal operation**.

```
List<String> result = names.stream()
    .filter(n -> n.startsWith("A"))
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
```

Intermediate: `filter`, `map`, `sorted`, `distinct`. Terminal: `collect`, `forEach`, `count`, `reduce`. Streams don't modify the source, and a stream can be consumed only once.

Part 8: JVM and Memory

Q14. Stack vs heap memory?

Show answer:

- **Stack:** per thread; holds method frames, local variables, and references; freed automatically when the method returns; fast; overflow gives `StackOverflowError`.
- **Heap:** shared; holds all objects; managed by the garbage collector; overflow gives `OutOfMemoryError`.

`String s = new String("a");` → the reference `s` is on the stack, the object is on the heap.

Want the rest?

You have just worked through 14 of the **44 questions** in the full Prep Trek Java Interview Q&A Bank.

The Premium Edition adds:

- Core Java fundamentals the free set skips: `==` vs `.equals()`, `static`, `final`, access modifiers, wrappers
- Exceptions deep-dive, including `finally` edge cases and try-with-resources
- The full Collections set: `HashMap` vs `ConcurrentHashMap`, `TreeSet`, `Comparable` vs `Comparator`, fail-fast iterators
- **Multithreading:** synchronization, `volatile`, deadlock, thread pools (the section that decides many offers)
- Java 8+: Optional, `map` vs `flatMap`
- JVM internals and garbage collection
- **Output-prediction drill:** the tricky “what does this print?” questions
- Coding-problem table and a self-scoring sheet to track readiness

👉 [Get the full Java Interview Q&A Bank on Topmate](#)

Want someone to listen to your answers and tell you what to fix? [Book a 1:1 mock interview](#) or [join the community](#).

Prep Trek: Java Interview Q&A, Free Sample.

Prep Trek: Mock Interview Practice Kit (Free Sample)

Who this is for: Freshers and early-career candidates preparing for Software Engineer, SDE, and Associate Software Engineer roles.

Try every question out loud *before* reading the answer below it.

1. How to Use This Kit

1. **Always answer out loud.** Silent thinking hides the exact problem interviewers test: can you explain while you think?
2. **Use a timer.** Pressure is part of the skill.
3. **Record yourself** (phone audio is fine). Listen back once per week.
4. **Log every miss** in the Mistakes Tracker (your own notes). Review it before your next mock.
5. **Do not skip the “say it out loud” step** in the DSA protocol below, even when practicing alone.

Scoring each mock

Score yourself (or have your partner score you) using the rubric in the premium scorecard. A mock is “passed” when you score **3 or higher on every row**. Aim for three passed mocks in a row before applying.

2. The 60-Minute Mock Structure

Time	Round	What happens
0-5 min	Intro	“Tell me about yourself” (30-45 sec), then small talk
5-35 min	DSA	1 medium or 2 easy problems, coded, with complexity discussion
35-50 min	CS fundamentals	8-10 rapid-fire questions (OOP, DBMS, OS, CN)
50-57 min	Project	Deep dive on one project
57-60 min	Close	Your questions for the interviewer

Service-company interviews lean toward the CS fundamentals and project blocks. Product-company interviews lean toward DSA. Adjust the time split to match your target.

3. Try Two DSA Mock Problems

The 7-step protocol (use it on every problem)

1. **Repeat the problem** in your own words.
2. **Ask clarifying questions:** input size? duplicates? negative numbers? empty input?
3. **Give an example** and walk through it by hand.
4. **State the brute force** and its complexity, even if it is bad.
5. **Optimize**, and say *why* it is better.
6. **Code** while narrating.
7. **Test** with a normal case, an edge case (empty, single element), and a tricky case.

Interviewers rate steps 2, 4, and 7 higher than most candidates expect.

Problem 1: Two Sum (Easy, 15 min)

Given an array of integers and a target, return the indices of the two numbers that add up to the target. Exactly one solution exists. You may not use the same element twice.

Example: `nums = [2, 7, 11, 15], target = 9` → `[0, 1]`

Hints:

- **Hint 1:** For each number, what exact value would you need to find to finish the pair?
- **Hint 2:** Searching an array for that value is slow. What structure gives you fast lookup?
- **Hint 3:** You can build the lookup *while* scanning, in one pass.

Show answer:

Brute force: two nested loops, check every pair. Time $O(n^2)$, space $O(1)$.

Optimal: one pass with a hash map from value → index. For each number `x`, check whether `target - x` is already in the map. If yes, return both indices. If no, store `x`.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
```

Complexity: Time $O(n)$, space $O(n)$.

Follow-ups interviewers ask:

- *What if the array is sorted?* Two pointers from both ends, $O(n)$ time, $O(1)$ space.
- *What if there are multiple valid pairs?* Collect results; use a set to avoid duplicate pairs.
- *What if memory is tight?* Sort + two pointers (but you lose original indices unless you store them).

Common mistake: checking and inserting in the wrong order, which lets an element pair with itself.

Problem 2: Valid Parentheses (Easy, 15 min)

Given a string containing only `()[]{}` , determine whether it is valid. Brackets must close in the correct order.

Examples: `"()[]{}"` → true, `"()"` → false, `"([])"` → false

Hints:

- **Hint 1:** The *most recent* unmatched opening bracket must be closed first. Which data structure behaves that way?
- **Hint 2:** What should happen when you see a closing bracket and the stack is empty?

Show answer:

Use a stack. Push opening brackets. On a closing bracket, the top of the stack must be its matching opener; otherwise return false. At the end the stack must be empty.

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}', '(': ')'}
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

Complexity: Time $O(n)$, space $O(n)$.

Edge cases to test: empty string, a single opener "(", a single closer ")", "({)".

Follow-up: Can you do it in $O(1)$ space if only one bracket type exists? Yes, use a counter (decrement on closer; fail if it goes negative; must end at 0).

4. Three CS Fundamentals Questions

Aim for 30 to 60 seconds per answer. If you run longer, you are probably reciting a textbook.

Q1. Explain encapsulation, inheritance, polymorphism, and abstraction with a real example.

Show answer:

Use a ride-hailing app.

- **Encapsulation:** a `Ride` object keeps fare and driver location private; other code uses methods like `getFare()`.
- **Inheritance:** `Driver` and `Rider` both extend `User`, sharing name, phone, and login logic.
- **Polymorphism:** `calculateFare()` behaves differently for `BikeRide` and `CarRide`.
- **Abstraction:** the app calls `bookRide()` without knowing how the matching algorithm works.

Q2. INNER JOIN vs LEFT JOIN?

Show answer:

INNER returns only rows with a match in both tables. LEFT returns every row from the left table, with NULLs where the right table has no match. Example: employees LEFT JOIN departments still shows an employee with no department.

Q3. What happens when you type a URL and press Enter?

Show answer:

1. Browser checks its cache, then DNS resolves the domain to an IP.
2. TCP three-way handshake with the server.
3. For HTTPS, a TLS handshake sets up encryption.
4. Browser sends an HTTP request.
5. Server processes it and returns a response.
6. Browser parses HTML, fetches CSS/JS/images, and renders the page.

You don't need every detail. A clear, ordered flow scores well.

5. Behavioral Round: The STAR Method

The STAR template

- **Situation:** one sentence of context.
- **Task:** what you were responsible for.
- **Action:** what **you** did (use "I", not "we"). This should be the longest part.
- **Result:** the outcome, with a number if possible, plus what you learned.

Keep answers under **90 seconds**.

Worked example: “Tell me about a difficult bug.”

S: In my final-year project, our app randomly logged users out. **T:** I was responsible for the authentication module, so I had to find the cause. **A:** I added logging around token creation and noticed the logouts happened about one hour after login, matching the token expiry. The frontend never refreshed the token. I implemented a refresh call before expiry and wrote a test that simulated a clock skip. **R:** Random logouts dropped to zero in testing. I learned to log early and to check time-based behavior first when a bug looks random.

Want the full kit?

You have just done 2 of the 6 DSA mock problems and 3 of the 16 CS fundamentals questions.

The Premium Edition adds:

- 4 more DSA problems (linked lists, cycle detection, trees, binary search traps) with graded hints and follow-ups
- 13 more CS fundamentals answers across OOP, DBMS, OS and Networks
- The project deep-dive question bank and answer template
- 12 behavioral questions to rehearse, plus questions to ask the interviewer
- The **interviewer scorecard**, peer-mock script and solo-mock protocol
- A mistakes tracker and the **4-week mock schedule** with a final readiness checklist

👉 [Get the full kit on Topmate](#)

Want a person to listen and give feedback? [Book a 1:1 mock review](#) or [join the community](#).

Prep Trek: Mock Interview Practice Kit, Free Sample.